

Golang — Amaliy masalalar

I bo'lim. Map (3 ta masala)

Masala 1. So'zlar chastotasi va TOP-N

Berilgan matnli qatorni so'zlarga ajratib, har bir so'z necha marta uchraganini `map[string]int` da hisoblang. So'zlar bo'sh joy bilan ajratilgan; katta-kichik harf farqlanmasin (barchasi kichik harfga keltirilsin) va so'z oxiridagi tinish belgilari (nuqta, vergul, undov, so'roq) tashlab yuborilsin.

So'ngra eng ko'p uchragan 3 ta so'zni chastotasi bo'yicha kamayish tartibida chiqaring. Agar ikkita so'zning chastotasi teng bo'lsa, ular alifbo tartibida joylashsin.

Kirish:

```
"Go tez, Go sodda. Go yaxshi til, til va yana til!"
```

Chiqish:

```
go: 3
til: 3
tez: 1
```

Eslatma: `map` ustidan `range` qilish tartibi Go'da tasodifiy — shuning uchun natijani chiqarishdan oldin `slice`'ga ko'chirib saralashingiz kerak bo'ladi.

Masala 2. Ikki tomonlama indeks (map ichida map)

Talabalar va ularning fanlar bo'yicha baholari `map[string]map[string]int` ko'rinishida berilgan (tashqi kalit — talaba ismi, ichki kalit — fan nomi, qiymat — baho).

```
baholar := map[string]map[string]int{
    "Ali":  {"matematika": 5, "fizika": 4, "tarix": 3},
    "Vali": {"matematika": 3, "fizika": 5},
    "Guli": {"matematika": 5, "tarix": 5, "fizika": 4},
}
```

Quyidagi vazifalarni bajaring:

a) Strukturani teskari aylantiring: fan nomi bo'yicha o'sha fanni o'qiyotgan talabalar va ularning baholari chiqadigan yangi `map` hosil qiling — `map[string]map[string]int` (tashqi kalit endi fan nomi).

b) Har bir fan bo'yicha o'rtacha bahoni hisoblab, `map[string]float64` ga yig'ing.

c) Barcha fanlardan faqat 5 baho olgan talabalar ro'yxatini (`slice` ko'rinishida) qaytaring. Agar bunday talaba bo'lmasa, bo'sh `slice` qaytsin.

Diqqat: ichki `map`'ga yozishdan oldin `uni make` bilan yaratishni unutmang — aks holda nil `map` ga yozishga urinib `panic` olasiz.

Masala 3. Ikki map'ni solishtirish va birlashtirish

Ikkita `map[string]int` berilgan — eski va yangi (masalan, omborxonadagi mahsulotlar va ularning soni). Ular o'rtasidagi farqni aniqlaydigan funksiya yozing. Funksiya uchta natija qaytarsin:

a) qo'shilgan — faqat yangi `map`'da bor kalitlar; b) o'chirilgan — faqat eski `map`'da bor kalitlar; c) o'zgargan — ikkalasida ham bor, lekin qiymati farq qiladigan kalitlar. Uchinchi natijada har bir kalit uchun qiymat qancha o'zgarganini (yangi — eski) ko'rsating.

```
eski := map[string]int{"olma": 10, "anor": 5, "uzum": 7}
yangi := map[string]int{"olma": 15, "uzum": 7, "shaftoli": 3}
```

Chiqish:

```
Qo'shilgan:  [shaftoli]
O'chirilgan: [anor]
O'zgargan:   map[olma:+5]
```

Kalit mavjudligini tekshirishda albatta ikki qiymatli formani (`v, ok := m[k]`) ishlatish — qiymati 0 bo'lgan kalit bilan umuman yo'q kalitni farqlay olishingiz kerak.

II bo'lim. Data type conversion (2 ta masala)

Masala 4. Xavfsiz konvertor

Qatorlar slice'i berilgan, unda turli xil ma'lumotlar bor:

```
data := []string{"42", "3.14", "true", "-17", "salom", "0", "99.99", "false", "1e3"}
```

Shu slice'ni bir marta aylanib chiqib, har bir elementni mos turga aylantirish va natijalarni turlar bo'yicha alohida slice'larga ajrating: butun sonlar (`[]int`), o'nlik sonlar (`[]float64`), mantiqiy qiymatlar (`[]bool`) va aylantirib bo'lmaganlar (`[]string`).

`strconv` paketidan foydalaning. Tekshirish tartibiga e'tibor bering: avval butun son, keyin float, keyin bool — chunki "42" ni `ParseFloat` ham muvaffaqiyatli o'qiy oladi, lekin u butun sonlar ro'yxatiga tushishi kerak. Oxirida har bir guruhni va topilgan xatolar sonini chiqaring.

Masala 5. Turlar orasidagi tuzoqlar

Quyidagi har bir band uchun natijani oldindan qog'ozga yozing, keyin kodni ishga tushirib tekshiring va nima uchun shunday bo'lganini izohlang:

```
a) var x int32 = 130
   var y int8 = int8(x)
   fmt.Println(y)

b) var a float64 = -3.99
   fmt.Println(int(a))

c) n := 65
   fmt.Println(string(rune(n)))
   fmt.Println(strconv.Itoa(n))

d) s := "salom"
   fmt.Println(len(s), len([]rune(s)))
   t := "caлom" // kirill harflar bilan
   fmt.Println(len(t), len([]rune(t)))

e) var u uint8 = 0
   u--
   fmt.Println(u)
```

Shundan so'ng `SafeToInt8(x int32) (int8, error)` funksiyasini yozing: agar `x` qiymati `int8` chegarasiga (`-128 ... 127`) sig'masa, aylantirish o'rniga tushunarli xato qaytarsin; sig'sa — aylantirilgan qiymatni qaytarsin.

III bo'lim. Anonim funksiyalar (5 ta masala)

Masala 6. Hisoblagich generatori (closure)

Hisoblagich(boshlangich int) func() int nomli funksiya yozing. U anonim funksiya qaytarsin; o'sha qaytgan funksiya har chaqirilganda hisoblagich qiymatini 1 ga oshirib, yangi qiymatni qaytarsin.

```
a := Hisoblagich(0)
b := Hisoblagich(100)

fmt.Println(a(), a(), a()) // 1 2 3
fmt.Println(b(), b())      // 101 102
fmt.Println(a())           // 4
```

Asosiy savol: a va b nima uchun bir-biriga ta'sir qilmaydi? Javobini kod izohida yozing.

Qo'shimcha: Hisoblagich2(boshlangich int) (func() int, func() int, func()) yozing — u uchta anonim funksiya qaytarsin: oshirish, kamaytirish va boshlang'ich holatga qaytarish. Uchalasi bitta umumiy o'zgaruvchi ustida ishlasin.

Masala 7. Sikl ichidagi klassik tuzoq

Quyidagi kod nima chop etadi? Nima uchun?

```
funksiyalar := []func(){}

for i := 1; i <= 3; i++ {
    funksiyalar = append(funksiyalar, func() {
        fmt.Print(i, " ")
    })
}

for _, f := range funksiyalar {
    f()
}
```

a) Kodni Go 1.22 dan oldingi versiyalarda ishga tushirsangiz natija qanday bo'ladi? Go 1.22 va undan keyingi versiyalarda-chi? Farqning sababini tushuntiring.

b) Eski Go versiyalarida ham to'g'ri ishlashi uchun kodni ikki xil usulda tuzating: (1) sikl ichida yangi lokal o'zgaruvchi hosil qilib, (2) anonim funksiya parametr sifatida uzatib.

c) Xuddi shu tuzoq defer bilan qanday namoyon bo'ladi? Sikl ichida defer func(){ ... }() yozib, misol keltiring.

Masala 8. O'z filter/map/reduce'ingizni yozing

Anonim funksiyalarni parametr sifatida qabul qiladigan uchta funksiya yozing:

```
func Filter(sonlar []int, shart func(int) bool) []int
func Map(sonlar []int, ozgartir func(int) int) []int
func Reduce(sonlar []int, boshlangich int, birlashtir func(int, int) int) int
```

So'ngra ularni faqat anonim funksiyalar bilan chaqirib, quyidagi vazifani yechning: []int{1..20} slice'idan juft sonlarni ajrating, har birini kvadratga ko'taring va yig'indisini toping.

```
natija := Reduce(
    Map(
        Filter(sonlar, func(n int) bool { ... }),
        func(n int) int { ... },
    ),
    0,
```

```
func(a, b int) int { ... },
)
```

Qo'shimcha: Filter ga o'tkaziladigan shartni ham qaytaruvchi funksiya yozing — Kopaytuvchisi(k int) func(int) bool, u "k ga bo'linadimi?" degan shartni tekshiradigan anonim funksiya qaytarsin. Uni Filter(sonlar, Kopaytuvchisi(3)) ko'rinishida ishlating.

Masala 9. Memoizatsiya (keshlovchi o'ram)

Sekin ishlaydigan funksiya bor deylik — masalan rekursiv Fibonachchi. Uni tezlashtiruvchi Memo(f func(int) int) func(int) int funksiyasini yozing: u anonim funksiya qaytarsin, o'sha funksiya ilgari hisoblangan natijalarni map[int]int keshida saqlab, takroriy chaqiruvda qayta hisoblamasdan tayyor javobni bersin.

```
sekin := func(n int) int {
    time.Sleep(300 * time.Millisecond)
    return n * n
}

tez := Memo(sekin)

fmt.Println(tez(9)) // ~300ms kutadi, 81
fmt.Println(tez(9)) // darhol, 81 (keshdan)
fmt.Println(tez(4)) // ~300ms kutadi, 16
```

Har bir chaqiruv qancha vaqt olganini time.Since bilan o'lchab chiqaring. Qo'shimcha: keshda nechta qiymat borligini qaytaradigan ikkinchi funksiyani ham birga qaytaring — func Memo(f func(int) int) (func(int) int, func() int).

Masala 10. Middleware zanjiri

Quyidagi ikkita turni e'lon qiling:

```
type Amal func(matn string) string
type Orab func(Amal) Amal
```

So'ngra anonim funksiyalar yordamida bir nechta o'ram (middleware) yozing: har biri asosiy amalni bajarishdan oldin yoki keyin qo'shimcha ish qilsin. Kamida uchta bo'lsin — masalan:

a) Log — amaldan oldin kirish qiymatini, keyin natijani chop etadi; b) Vaqt — amal necha vaqt bajarilganini o'lchaydi; c) Katta — natijani katta harflarga o'giradi.

Ularni birlashtiruvchi Zanjir(a Amal, oramlar ...Orab) Amal funksiyasini yozing — u barcha o'ramlarni ketma-ket qo'llab, yagona Amal qaytarsin. O'ramlar ro'yxatdagi tartibda qo'llanishiga e'tibor bering (birinchi berilgani eng tashqarida bo'lsin).

```
asosiy := func(matn string) string {
    return matn + "!"
}

yakuniy := Zanjir(asosiy, Log, Vaqt, Katta)
fmt.Println(yakuniy("salom"))
```

Chiqishda o'ramlarning qaysi tartibda ishlaganini kuzating va nima uchun aynan shunday tartibda bo'lganini izohlab bering.